HMRC: Heterogeneous Multi-Representative Centroids for Improved Vector Database Routing

Kishore Pulagam
Northwestern University
Evanston, IL, USA
kishore.pulagam@kellogg.northwestern.edu

Abstract

Vector databases partition vectors into segments and use centroids to route queries. When segments contain vectors from multiple topics/domains, the mean centroid fails to represent any topic well, leading to routing errors. Our experiments on MS MARCO (100K passages) show that routing recall drops from 100% on coherent segments to 64.4% when segments mix 3 topics, and down to 14.6% on random segments. This is often the case in enterprise content management and RAG systems, rendering the simple centroid based routing inefficient.

We address this with HMRC (Heterogeneous Multi-Representative Centroids), which uses k-means clustering to generate multiple centroids per segment. With 3 centroids per segment, routing recall improves from 64.4% to 81.2%—a 26% relative gain—with only 3x centroid storage overhead. Performance on coherent segments remains at 100%, and the approach generalizes across segment sizes from 100 to 2000 vectors. HMRC is a drop-in replacement for any IVF-style index and requires no changes to segment structure or search logic.

CCS Concepts

• **Information systems** → **Information retrieval**.

Keywords

vector databases, approximate nearest neighbor search, IVF indexing, embeddings, RAG systems

1 Introduction

Vector databases are now essential infrastructure for retrieval-augmented generation (RAG) systems, semantic search, and recommendation engines. As embedding models improve and corpora grow to billions of vectors, efficient approximate nearest neighbor (ANN) search becomes critical.

To scale, most vector databases use segment-based architectures. The corpus is partitioned into segments (also called clusters or partitions), and each segment is represented by a centroid—typically the mean of its vectors. At query time, the system first compares the query to all centroids, identifies the most promising segments, and searches only within those segments. This two-phase approach reduces search complexity from $O(n)$ to $O(n/m + m)$ where n is corpus size and m is the number of segments.

The routing step is critical. If the correct segment is not among the top candidates, the target vector cannot be found regardless of how thoroughly the selected segments are searched. Routing accuracy directly bounds retrieval recall.

1.1 The Problem

There is a gap in how these systems handle heterogeneous segments. When a segment contains vectors from multiple topics, the mean centroid lands between topics and may not be close to any of them. A query about topic A might be closer to another segment's centroid than to the mean of a segment that actually contains topic A content.

Consider a concrete example. A segment contains 500 vectors: 200 about machine learning, 150 about cooking, and 150 about sports. The mean centroid sits somewhere in the middle of these three topic clusters in embedding space. A query about "gradient descent optimization" is much closer to the machine learning cluster than to this mean, so it may route to a different segment that happens to have more concentrated ML content.

This is not a contrived scenario. Heterogeneous segments arise naturally in production systems:

- **Time-based ingestion:** Documents are batched by arrival time, not topic. A day's worth of news articles spans many subjects.
- **Multi-tenant systems:** Vectors are partitioned by customer or source, not by semantic content.
- **Incremental updates:** New documents get appended to existing segments rather than triggering a full re-clustering.
- **Imperfect initial clustering:** Even k-means clustering does not produce perfectly homogeneous segments.

1.2 Our Contribution

We make three contributions:

(1) **Problem quantification.** We measure the impact of segment heterogeneity on routing accuracy using real data. Mixing just 3 topics per segment drops recall from 100% to 64.4%.
(2) **HMRC.** We propose Heterogeneous Multi-Representative Centroids, which uses k-means within each segment to generate multiple centroids that capture internal structure.
(3) **Evaluation.** We show HMRC improves routing recall by 26% on realistic heterogeneous segments, maintains 100% recall on coherent segments, and generalizes across segment sizes.

The rest of the paper is organized as follows. Section 2 covers related work. Section 3 formalizes and quantifies the problem. Section 4 describes HMRC. Section 5 presents experiments. Section 6 discusses limitations and practical considerations. Section 7 concludes.

2 Related Work

2.1 Inverted File Indexing

Inverted File (IVF) indexing is the dominant approach for scaling vector search. FAISS [1] popularized this method, which clusters vectors using k-means and routes queries to the nearest cluster centroids. Milvus [4], Pinecone, Weaviate, and Qdrant all use variants of this architecture.

The standard way to handle routing errors is multi-probe search: instead of searching only the single nearest cluster, search the top-*nprobe* clusters. This improves recall but increases search cost linearly with *nprobe*. Importantly, multi-probe does not fix the underlying representation problem—it just searches more segments to compensate.

2.2 Spilling and Replication

Google's ScaNN [2] introduced SOAR (Spilling with Orthogonally Augmented Residuals), which assigns each vector to multiple clusters. This improves routing recall at the cost of 2-3x storage for the vectors themselves.

The key difference from HMRC is the level of operation. SOAR replicates vectors (expensive), while HMRC replicates centroids (cheap). For a system with 10,000 segments of 1,000 vectors each, SOAR would store 20-30 million vector copies, while HMRC-3 stores only 30,000 centroids.

2.3 Graph-Based Methods

HNSW [3] and other graph-based methods avoid centroids entirely by building a navigable graph over the vectors. These methods achieve excellent recall but have higher memory overhead (storing graph edges) and are harder to distribute across machines. They also do not support the segment-based architecture that many production systems require for data management.

2.4 Learned Indexes

Recent work explores learning routing functions rather than using geometric centroids [6]. While promising, these approaches require training data and may not generalize across domains. HMRC is training-free and domain-agnostic.

2.5 Position of Our Work

None of the above approaches directly address segment-level centroid representation. Multi-probe accepts poor centroids and compensates with more search. SOAR improves representation at the vector level. Graph methods avoid the problem by avoiding centroids. HMRC improves representation at the segment level with minimal overhead.

3 Problem: Centroid Degradation

3.1 Formal Setup

Let $\mathcal{D} = \{\mathbf{v}_1, \ldots, \mathbf{v}_n\}$ be a corpus of n vectors in $\mathbb{R}^d$. The corpus is partitioned into m segments $\{S_1, \ldots, S_m\}$ where $\bigcup_j S_j = \mathcal{D}$ and $S_i \cap S_j = \emptyset$ for $i \neq j$.

Table 1: Routing recall across segment types and k values

Segment Type	@1	@5	@10	@20
Coherent	95.4%	100%	100%	100%
Semi-3 (3 topics)	13.3%	45.5%	64.4%	77.1%
Semi-5 (5 topics)	8.2%	30.9%	49.1%	66.2%
Random	2.9%	8.8%	14.6%	24.1%

Each segment S_j is represented by a centroid $\mathbf{c}_j$, typically the mean:

$$\mathbf{c}_j = \frac{1}{|S_j|} \sum_{\mathbf{v} \in S_j} \mathbf{v} \tag{1}$$

Given a query $\mathbf{q}$, routing selects the top-k segments by centroid distance:

$$\text{Route}(\mathbf{q}, k) = \underset{j \in [m]}{\text{argmin-}k} \; \|\mathbf{q} - \mathbf{c}_j\| \tag{2}$$

Routing Recall@k is the fraction of queries where the ground-truth segment (containing the target vector) appears in the top-k routed segments.

3.2 Why Mean Centroids Fail

Consider a heterogeneous segment $S = S_A \cup S_B$ containing vectors from two topic clusters with means $\boldsymbol{\mu}_A$ and $\boldsymbol{\mu}_B$. The segment mean is:

$$\mathbf{c} = \frac{|S_A|}{|S|}\boldsymbol{\mu}_A + \frac{|S_B|}{|S|}\boldsymbol{\mu}_B \tag{3}$$

This centroid lies on the line between $\boldsymbol{\mu}_A$ and $\boldsymbol{\mu}_B$. For a query $\mathbf{q}$ near topic A:

$$\|\mathbf{q} - \mathbf{c}\| > \|\mathbf{q} - \boldsymbol{\mu}_A\| \tag{4}$$

If another segment S' has a centroid $\mathbf{c}'$ closer to the topic A region, then $\|\mathbf{q} - \mathbf{c}'\| < \|\mathbf{q} - \mathbf{c}\|$ and the query routes to S' instead of S, missing the target.

The problem worsens with more topics. With t topics mixed in a segment, the mean centroid is pulled toward the center of the topic constellation, potentially far from all individual topics.

3.3 Empirical Validation

We validated this on MS MARCO passages, a standard information retrieval benchmark. We embedded 100,000 passages using all-MiniLM-L6-v2 (384 dimensions) and created segments under four conditions:

- **Coherent:** Segments formed by k-means clustering (each segment contains one topic)
- **Semi-t:** Each segment mixes vectors from t random topic clusters
- **Random:** Vectors assigned uniformly at random

Figure 1 shows the results. Routing recall at $k = 10$ drops from 100% on coherent segments to 64.4% on Semi-3 (3 topics mixed), 49.1% on Semi-5, and just 14.6% on random segments.

Table 1 shows recall at different k values. Even at $k = 20$, random segments achieve only 24.1% recall—the routing problem is fundamental, not just a matter of searching more segments.

We also measured intra-segment distance (mean distance from vectors to their segment centroid) as a proxy for heterogeneity.

fig1_problem_validation.png

Figure 1: Routing recall degrades as segments become more heterogeneous. Coherent segments achieve 100% recall; mixing just 3 topics drops recall to 64.4%.

Coherent segments have mean intra-distance of 0.84, Semi-3 has 0.97, and Random has 0.99. Higher intra-distance correlates with worse routing.

4 HMRC: Our Approach

The core idea of HMRC is simple: instead of one centroid per segment, use multiple centroids that capture the segment's internal structure. We use k-means clustering within each segment to discover these centroids.

4.1 Index Construction

For each segment S_j, we run k-means with r clusters (we use $r = 3$ as default). The resulting cluster centers become the segment's representatives:

$$R_j = \text{KMeans}(S_j, r).\text{centers} \tag{5}$$

We store all representatives in a flat list $R = \bigcup_j R_j$ along with a mapping $M : R \to [m]$ from each representative to its segment ID.

For small segments (fewer than $2r$ vectors), k-means would be unreliable, so we fall back to using just the mean.

4.2 Query Routing

Given a query $\mathbf{q}$, we compute distances to all representatives (not just one per segment), find the nearest ones, and return segment IDs after deduplication.

4.3 Why K-Means Works

K-means clustering naturally discovers the topic structure within a heterogeneous segment. If a segment contains vectors from 3 topics, k-means with $k = 3$ will find centroids near each topic cluster. This gives queries from any topic a nearby representative to match.

When a segment is already coherent (one topic), k-means still works: it finds r centroids that are all close together, covering the same region. Routing still succeeds because any of these centroids will be near any query from that topic.

Algorithm 1 HMRC Index Construction

Require: Segments $\{S_1, \ldots, S_m\}$, number of representatives r
Ensure: Representative set R, segment mapping M

```
R ← ∅, M ← ∅
for j = 1 to m do
    if |S_j| < 2r then
        R_j ← {mean(S_j)}
    else
        R_j ← KMeans(S_j, r).centers
    end if
    R ← R ∪ R_j
    for each c ∈ R_j do
        M[c] ← j
    end for
end for
return R, M
```

Algorithm 2 HMRC Query Routing

Require: Query $\mathbf{q}$, representatives R, mapping M, parameter k
Ensure: Top-k segment IDs

```
Compute d(q, c) for all c ∈ R
Sort representatives by distance (ascending)
result ← [], seen ← ∅
for each c in sorted order do
    s ← M[c]
    if s ∉ seen then
        Append s to result
        Add s to seen
    end if
    if |result| ≥ k then
        break
    end if
end for
return result
```

4.4 Complexity Analysis

Storage. Baseline stores m centroids. HMRC stores rm centroids (assuming all segments use r representatives). For 10,000 segments with 384-dimensional vectors (float32):

- Baseline: $10K \times 384 \times 4$ bytes = 15 MB
- HMRC-3: $30K \times 384 \times 4$ bytes = 45 MB

This is negligible compared to storing the vectors themselves (e.g., 10M vectors would require 15 GB).

Index Build Time. K-means within each segment adds $O(r \cdot |S_j| \cdot d \cdot t)$ per segment, where t is the number of iterations. In practice, build time increases by 20-30x over computing means, but

fig2_hmrc_solution.png

Figure 2: HMRC improves routing recall on Semi-3 segments. HMRC-3 achieves 81.2% recall compared to 64.4% baseline (+26% relative gain).

this is a one-time cost. For 100K vectors in 200 segments, HMRC-3 builds in under 1 second on a laptop.

Query Time. Routing computes rm distances instead of m, a factor of r increase. For $r = 3$ and $m = 10K$, this is 30K distance computations—still negligible compared to searching within segments.

5 Experiments

5.1 Setup

Dataset. MS MARCO passages (100,000 passages) embedded with all-MiniLM-L6-v2 (384 dimensions). We created 200 segments of 500 vectors each under different mixing conditions.

Queries. 2,000 queries sampled randomly from the corpus. Ground truth is the segment containing each query's source vector.

Metric. Routing Recall@k: fraction of queries where the ground-truth segment appears in the top-k routed segments.

Baselines.

- **Mean Centroid:** Standard single centroid per segment
- **HMRC-r:** Our method with r representatives per segment

5.2 Main Results

Figure 2 and Table 2 show HMRC performance on Semi-3 segments (the realistic mixed case).

HMRC-3 improves Recall@10 from 64.4% to 81.2%, a relative gain of 26%. The improvement is consistent across k values: +13.6% absolute at $k = 1$, +27.2% at $k = 5$, +16.8% at $k = 10$.

Going beyond 3 representatives provides diminishing returns. HMRC-5 and HMRC-7 achieve similar recall to HMRC-3 while adding more overhead. This makes sense: Semi-3 segments contain 3 topics, so 3 representatives suffice.

Table 2: HMRC performance on Semi-3 segments at different k values

Method	@1	@5	@10	@20	Overhead
Baseline	13.3%	45.5%	64.4%	77.1%	1x
HMRC-2	20.9%	57.4%	72.1%	81.4%	2x
HMRC-3	26.9%	72.7%	81.2%	84.3%	3x
HMRC-5	25.3%	70.9%	80.7%	85.6%	5x
HMRC-7	28.3%	71.1%	80.9%	86.3%	7x

fig6_full_comparison.png

Figure 3: HMRC performance across segment types. HMRC maintains 100% recall on coherent segments while improving Semi-3 from 64.4% to 81.2%.

Table 3: Routing Recall@10 across segment types

Method	Coherent	Semi-3	Random
Baseline	100.0%	64.4%	14.6%
HMRC-3	100.0%	81.2%	16.2%
HMRC-5	100.0%	80.7%	22.1%

5.3 Generalization Across Segment Types

A critical question is whether HMRC hurts performance on segments that are already coherent. Table 3 and Figure 3 show this is not the case.

On coherent segments, HMRC maintains 100% recall. The k-means centroids end up clustered together when the segment is tight, so routing still works. HMRC adds no harm to already-good segments.

fig3_topics_ablation.png

Figure 4: HMRC benefit across heterogeneity levels. The gap between baseline and HMRC-3 widens as segments become more heterogeneous.

Table 4: Recall@10 vs. topics per segment

Topics	Baseline	HMRC-3	Relative Gain
1 (Coherent)	100.0%	100.0%	0%
2	73.5%	80.0%	+9%
3	64.4%	81.2%	+26%
5	49.1%	62.7%	+28%
7	40.0%	48.9%	+22%
10	30.9%	40.5%	+31%

On random segments, improvement is modest (14.6% to 16.2% for HMRC-3, 22.1% for HMRC-5). This is expected: random segments have no topic structure for k-means to discover. Such segments are uncommon in practice.

5.4 Effect of Heterogeneity Level

Figure 4 shows how HMRC benefit varies with the number of topics mixed per segment.

At low heterogeneity (1-2 topics), the baseline performs reasonably well and HMRC provides modest gains. As heterogeneity increases, both methods degrade, but HMRC degrades more slowly. The relative gain remains in the 22-31% range across heterogeneity levels.

5.5 Effect of Number of Representatives

Figure 5 shows how recall changes with the number of representatives r.

Performance improves sharply from $r = 1$ to $r = 3$, then plateaus. For Semi-3 segments (3 topics), $r = 3$ is sufficient to cover the topic structure. Adding more representatives beyond the number of topics does not help and only adds overhead.

This suggests a practical heuristic: set r to match the expected heterogeneity level. If segments are known to mix many topics, increase r; otherwise, $r = 3$ is a robust default.

fig4_representatives_ablation.png

Figure 5: Left: Recall vs. number of representatives. Performance plateaus at $r = 3$. Right: Storage overhead scales linearly with r.

Table 5: Effect of number of representatives on Semi-3 segments

Representatives	Recall@10	Overhead
1 (Baseline)	64.4%	1x
2	72.1%	2x
3	81.2%	3x
4	81.4%	4x
5	80.7%	5x
7	80.9%	7x
10	81.4%	10x

5.6 Effect of Segment Size

Figure 6 shows HMRC performance across segment sizes from 100 to 2000 vectors.

HMRC provides gains at all segment sizes. The relative improvement is largest for small segments (97% gain at size 100) and decreases for larger segments (5% at size 2000). This makes sense: smaller segments have less room to absorb topic diversity, so the mean centroid is more affected by mixing. Larger segments dilute the effect of any single topic.

fig5_segment_sizes.png

Figure 6: HMRC provides consistent gains across segment sizes, with larger relative improvements on smaller segments.

Table 6: Recall@10 vs. segment size (Semi-3 segments)

Size	Segments	Baseline	HMRC-3	Gain
100	1000	36.5%	71.9%	+97%
250	400	49.5%	77.4%	+56%
500	200	64.4%	81.2%	+26%
1000	100	69.0%	76.8%	+11%
2000	50	71.4%	74.7%	+5%

6 Discussion

6.1 When Does HMRC Help?

HMRC is most effective when segments have discoverable structure—distinct topic clusters that k-means can identify. This covers most production scenarios:

- Time-batched ingestion where documents from different topics arrive together
- Multi-tenant systems where each tenant has diverse content
- Incremental updates that mix new topics into existing segments

HMRC provides limited benefit on uniformly random segments because there is no structure to discover. However, such segments are rare in practice; real-world data almost always has some topic structure.

6.2 Choosing r

Based on our experiments:

- **Default:** $r = 3$ works well for typical heterogeneity
- **High heterogeneity:** $r = 5$ if segments are known to mix many topics
- **Low overhead:** Even $r = 2$ provides meaningful improvement (+12%)

A more sophisticated approach would adaptively choose r per segment based on detected heterogeneity (e.g., using silhouette score). We leave this for future work.

6.3 Limitations

Build time. HMRC increases index build time by 20-30x due to k-means clustering. For large corpora (billions of vectors), this may require optimization (e.g., mini-batch k-means, approximate clustering).

Updates. When vectors are added to a segment, the representatives should be recomputed. This is the same as the baseline (recompute mean) but r times more expensive.

Random segments. HMRC does not help when segments have no structure. Systems with truly random partitioning should consider different solutions.

6.4 Integration

HMRC is a drop-in replacement for existing IVF systems. The segment structure remains unchanged. The search logic within segments is unchanged. Only the centroid computation and routing logic change:

(1) Replace mean computation with k-means
(2) Store r centroids per segment instead of 1
(3) Route using all centroids with segment deduplication

Any system using centroid-based routing (FAISS, Milvus, etc.) can adopt HMRC with minimal code changes.

7 Conclusion

We identified a gap in vector database routing: heterogeneous segments are poorly represented by mean centroids, causing routing failures. On MS MARCO, mixing 3 topics per segment drops routing recall from 100% to 64.4%.

HMRC addresses this by using k-means to generate multiple centroids per segment. With $r = 3$, recall improves from 64.4% to 81.2%—a 26% relative gain—at 3x centroid storage cost. Coherent segments remain unaffected (100% recall), and the approach generalizes across segment sizes.

The fix is simple to implement and applies to any system using centroid-based routing. As vector databases become critical infrastructure for RAG and semantic search, improving routing accuracy directly improves end-to-end retrieval quality.

7.1 Future Work

Several directions remain:

- Adaptive r selection based on detected segment heterogeneity
- Incremental representative updates without full recomputation
- Combining HMRC with multi-probe for additional gains
- Evaluation on larger corpora (billions of vectors)

References

[1] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.

[2] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. Accelerating large-scale inference with anisotropic vector quantization. In *International Conference on Machine Learning*, pages 3887–3896, 2020.

[3] Yu A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2018.

[4] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 International Conference on Management of Data*, pages 2614–2627, 2021.

[5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

[6] Jingdong Wang, Ting Zhang, Jingkang Song, Nicu Sebe, and Heng Tao Shen. A survey on learning to hash. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(4):769–790, 2018.

[7] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011.

[8] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. Cache locality is not enough: High-performance nearest neighbor search with product quantization fast scan. *Proceedings of the VLDB Endowment*, 9(4):288–299, 2015.

[9] Dmitry Baranchuk, Artem Babenko, and Yury Malkov. Revisiting the inverted indices for billion-scale approximate nearest neighbors. In *Proceedings of the European Conference on Computer Vision*, pages 202–216, 2018.

[10] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The Faiss library. *arXiv preprint arXiv:2401.08281*, 2024.